APM 2025-2026 连续考核 1：参考解析

整理日期：2026 年 10 月 18 日

目录

I 课程知识题 **2**
Q.1 cudaMallocHost、cudaMalloc 与 malloc 2
Q.2 什么是合并访存 2
Q.3 三维 block、二维 grid 的全局编号 3
Q.4 哪个不是 CUDA 数据属性 3
Q.5 OpenMP 数据区域对应的 OpenACC 指令 3

II 多个 kernel 的先后依赖 **4**
Q.6 用 events 建立所需依赖 4

III Safe/Unsafe：先列条件，再判断 **5**
Q.7 tmp 是 pinned host 指针：需条件化判断 6
Q.8 把字节数用于 int 指针偏移 6
Q.9 默认 stream 保证 GPU 顺序，不保证 CPU 已可用 7
Q.10 两类独立错误：越界与跨流数据竞争 7

IV CUDA 求和归约 **8**
Q.11 共享内存归约和两次调用 8

V 用这张卷调整复习顺序 **9**

共 11 题，原卷注明时长 3 小时，未标分值。

独立练习题目 · 法语解析 · 原图与资料入口 · 完整复习讲义

I 课程知识题

Q.1 cudaMallocHost、cudaMalloc 与 malloc

题意：解释三种分配函数的用途和差别。

函数	分配的主要位置与性质	释放方式
`malloc(bytes)`	普通主机内存，通常可分页；适合 CPU 使用	`free(ptr)`
`cudaMallocHost(&ptr, bytes)`	主机页锁定内存，即 pinned memory；用于 CUDA 传输等	`cudaFreeHost(ptr)`
`cudaMalloc(&ptr, bytes)`	本题采用的模型中是 GPU 设备内存	`cudaFree(ptr)`

`cudaMallocHost` 把分配结果写入调用者传入的指针变量，因此传 `&ptr`；大小单位是字节。它不初始化内容。页锁定内存适合与异步 H2D/D2H 传输配合，能避免普通 pageable 内存所需的中间暂存，提供传输与计算重叠的条件；不保证任何程序都会变快。

与 Q.7 的联系：主机内存是否能由 kernel 直接访问，需要看映射和设备能力。不能把 pinned memory 一概认定为不可被 GPU 访问；也不能把它当成 `cudaMallocManaged` 分配的统一内存。这一点在 Q.7 单独讨论。资源有限时，过量锁页也可能影响系统。

Réponse courte : `cudaMallocHost` alloue une mémoire hôte verrouillée en pages, adaptée aux transferts CUDA asynchrones. `cudaMalloc` alloue une mémoire device, tandis que `malloc` alloue une mémoire hôte ordinaire. Les fonctions de libération correspondantes sont `cudaFreeHost`, `cudaFree` et `free`.

复习讲义对应第 2.4、2.11 节。

Q.2 什么是合并访存

题意：解释 accès coalescé 的含义。

同一 warp 中活跃线程对 global memory 的访问，如果落在邻近、适当对齐的地址区域，硬件通常能够用较少的内存事务服务这些访问。常见有利模式是相邻 lane 访问相邻数组元素：

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
value = a[i];
```

若访问变成 `a[i * stride]` 且 `stride` 很大，访问会分散到更多内存段。确切事务数还取决于类型大小、对齐、活跃线程和架构，不能无条件写成“32 个线程一定只产生一次事务”。合并访存是 warp 地址模式的性质，不等于线程间同步。

Réponse courte : Un accès coalescé regroupe les accès mémoire des threads d’ un warp en un petit nombre de transactions. Des threads voisins accédant à des éléments contigus et correctement alignés favorisent cette propriété.

复习讲义对应第 2.7 节。

Q.3 三维 block、二维 grid 的全局编号

题意：写出统一线程编号，明确展平约定。

采用 x 维最快、然后 y、最后 z 的顺序；先给 block 编号，再把每个 block 的所有线程连续排列：

$$
\begin{aligned}
b &= \mathrm{blockIdx.x} + \mathrm{GRID_X}\,\mathrm{blockIdx.y},\\
t &= \mathrm{threadIdx.x} + \mathrm{BLOCK_X}(\mathrm{threadIdx.y} + \mathrm{BLOCK_Y}\,\mathrm{threadIdx.z}),\\
B &= \mathrm{BLOCK_X}\,\mathrm{BLOCK_Y}\,\mathrm{BLOCK_Z},\\
g &= bB + t.
\end{aligned}
$$

```
size_t block_id = (size_t)blockIdx.y * gridDim.x + blockIdx.x;
size_t threads_per_block = (size_t)blockDim.x * blockDim.y * blockDim.z;
size_t local_id = threadIdx.x + (size_t)blockDim.x *
                 (threadIdx.y + (size_t)blockDim.y * threadIdx.z);
size_t global_id = block_id * threads_per_block + local_id;
```

`GRID_Y` 决定总块数和编号上界，不必单独出现在当前 block 的展平式中。总线程数为 `GRID_X*GRID_Y*B`，编号范围为 `0` 到总线程数减 1。网格是二维，因此不加入 `blockIdx.z` 项。

例如 `grid=(3,2)`、`block=(4,2,2)`、`blockIdx=(1,1)`、`threadIdx=(2,1,1)`，得到 `b=4`、`B=16`、`t=14`、`g=78`。如果题目要求三维数组地址，则需另按数据坐标展平；那不是这里的按块编号。

复习讲义对应第 1.10、1.11、4.2 节。

Q.4 哪个不是 CUDA 数据属性

答案：第 3 项 `__local__`。

CUDA 有 local memory 这个存储空间，但没有这里列出的 C/C++ 数据声明属性 `__local__`。普通线程私有变量可位于寄存器或 local memory，由编译器和具体使用方式决定。

`__device__` 可以用于设备变量声明，也可以作为设备函数属性；不能因为它常见于函数就选它。`__shared__` 用于共享内存变量，`__constant__` 用于常量存储空间中的变量。不要与 OpenCL 的 `__local` 混淆。

复习讲义对应第 2.1、2.3 节。

Q.5 OpenMP 数据区域对应的 OpenACC 指令

答案：

```
#pragma acc data copyin(a[0:8])
{
    /* Region using a on the accelerator. */
}
```

原题的 `target data map(to: a[0:8])` 对应结构化 OpenACC `data copyin` 区域。在通常的独立设备、没有既有映射的情形下，进入区域时建立数据并复制输入，离开时不把修改后的数据回传。`0:8` 表示从下标 0 开始的 8 个元素。

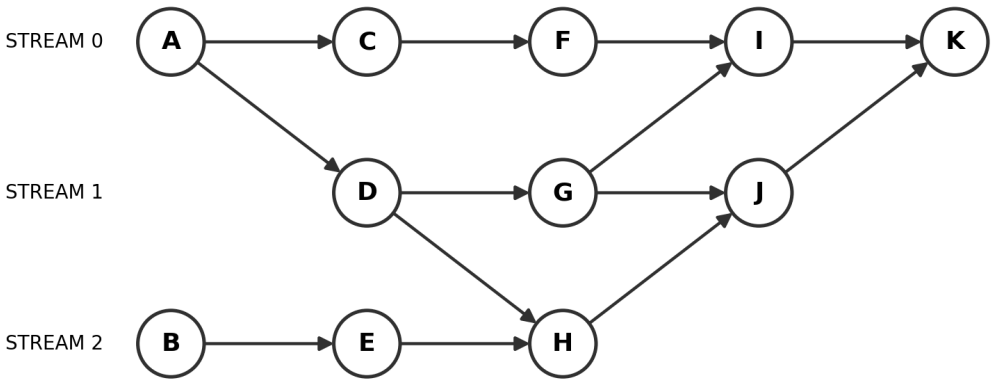
单独的 `data` 区域管理数据，本身不执行并行计算；区域内仍需相应计算指令。`enter data copyin` 属于另一种非结构化生命周期，不能不加说明就替换本题的结构化区域。

参考：OpenACC 官方编程指南。完整复习讲义第 3.13 节讲 OpenMP Target；本题补充其 OpenACC 对应写法。

II 多个 kernel 的先后依赖

Q.6 用 events 建立所需依赖

题意：在 3 个 stream 中调用 11 个 kernel，只加入依赖图要求的同步。



同一 stream 已保证提交顺序：

- `s[0]`：A → C → F → I → K。
- `s[1]`：D → G → J。
- `s[2]`：B → E → H。

还需保证 5 条跨 stream 依赖：**A → D、D → H、G → I、H → J、J → K**。不能用 CPU 的调用先后推定不同 stream 中的 GPU 完成先后。

以下代码只展示提交与同步，假设 kernel 定义、参数、网格及 block 大小已给出，所有 API 调用成功。实际答题可以用 `A<<<...>>>(...)` 表示相应 kernel 的参数；这里统一省略参数。

```
cudaStream_t s[3];
for (int i = 0; i < 3; ++i)
    cudaStreamCreateWithFlags(&s[i], cudaStreamNonBlocking);
cudaEvent_t eA, eD, eG, eH, eJ;
cudaEventCreateWithFlags(&eA, cudaEventDisableTiming);
cudaEventCreateWithFlags(&eD, cudaEventDisableTiming);
cudaEventCreateWithFlags(&eG, cudaEventDisableTiming);
cudaEventCreateWithFlags(&eH, cudaEventDisableTiming);
cudaEventCreateWithFlags(&eJ, cudaEventDisableTiming);

A<<<grid, block, 0, s[0]>>>();
cudaEventRecord(eA, s[0]);
B<<<grid, block, 0, s[2]>>>();
C<<<grid, block, 0, s[0]>>>();
F<<<grid, block, 0, s[0]>>>();

cudaStreamWaitEvent(s[1], eA, 0);
D<<<grid, block, 0, s[1]>>>();
cudaEventRecord(eD, s[1]);
G<<<grid, block, 0, s[1]>>>();
cudaEventRecord(eG, s[1]);

E<<<grid, block, 0, s[2]>>>();
cudaStreamWaitEvent(s[2], eD, 0);
H<<<grid, block, 0, s[2]>>>();
cudaEventRecord(eH, s[2]);

cudaStreamWaitEvent(s[0], eG, 0);
I<<<grid, block, 0, s[0]>>>();
cudaStreamWaitEvent(s[1], eH, 0);
J<<<grid, block, 0, s[1]>>>();
cudaEventRecord(eJ, s[1]);
cudaStreamWaitEvent(s[0], eJ, 0);
K<<<grid, block, 0, s[0]>>>();

// Before the CPU consumes results or resources are released:
cudaStreamSynchronize(s[0]);
for (int i = 0; i < 3; ++i) cudaStreamDestroy(s[i]);
cudaEventDestroy(eA);
cudaEventDestroy(eD);
cudaEventDestroy(eG);
cudaEventDestroy(eH);
cudaEventDestroy(eJ);
```

为何是这些等待?屏障放在消费者之前：D 等 A，H 等 D，I 等 G，J 等 H，K 等 J。每个生产者之后记录 event；本例中相关 record 均先于 wait 在主机端提交，避免等待一个尚未记录的 event。`cudaStreamWaitEvent` 给 GPU 队列添加依赖，不要求 CPU 为每条边停住。

“最少”的边界：在保留图中并行机会、不增加额外全局顺序的前提下，这 5 条跨流边不能由其他边推出。用每个生产者完成后记录 event、消费者等待的写法时，5 次 wait 恰好覆盖它们。只比较主机同步调用数，可以把更多工作串行化后少写几次等待，但会添加图中没有要求的依赖。

K 是所有节点的后继，所以本例最后等待 `s[0]` 的 K 完成，即覆盖了图中所有 kernel。不必为每条边调用 `cudaDeviceSynchronize()`。如果还有图外的任务、复制或其他输出使用者，需另外判断其完成条件。

参考：NVIDIA events、stream 等待 event。复习讲义对应第 2.11、4.13 节。

III Safe/Unsafe：先列条件，再判断

这些代码片段没有给出 kernel 函数体、完整错误处理与所有外围声明。以下按 `N>0`、尺寸乘法不溢出、合法 launch、kernel 只访问约定区间进行分析；不能凭省略的函数体编造结果。原题程序完整保留在独立题目文件中。

Q.7 tmp 是 pinned host 指针：需条件化判断

题意：一个 stream 中依次执行 H2D、kernel、D2H、同步，然后打印。kernel 额外收到由 `cudaMallocHost` 得到的 `tmp`。

已成立的顺序：所有操作都在 `stream[0]`；D2H 后的 `cudaStreamSynchronize(stream[0])` 保证打印前复制已完成。这里没有“少了 D2H 等待”的问题。

真正需要检查：`tmp` 的设备可访问性和初始化。传统分离内存的简化教学模型中，直接传主机指针给 kernel 往往被判为 unsafe；但这不是所有 CUDA 环境下的结论。在支持 UVA 的设备上，官方文档规定普通 `cudaMallocHost` 分配可直接由设备通过同值指针访问。

因此，**仅凭照片不能无条件判 unsafe，也不能证明 safe**：若设备支持相应访问，kernel 只把 `tmp` 当作已正确写入的工作区，且访问不越界，此片段的队列顺序是安全的；若 kernel 读取未初始化 `tmp`，或环境不允许直接访问该 host pointer，则不安全。把假设写出来，比只圈 safe/unsafe 更完整。

可移植的修正思路：若 `tmp` 用作设备工作区，用 `cudaMalloc` 分配并在读取前初始化；若要用映射主机内存，则明确确认设备能力与有效设备指针，并维持 CPU/GPU 间的访问顺序。

```
int *d_tmp = nullptr;
size_t bytes = (size_t)N * sizeof(int);
cudaMalloc((void **)&d_tmp, bytes);
// Only needed if the kernel reads the initial scratch values:
cudaMemsetAsync(d_tmp, 0, bytes, stream[0]);
kernel<<<grid, block, 0, stream[0]>>>(dtab, d_tmp, N);
// Queue the D2H copy, wait for the stream, then cudaFree(d_tmp).
```

这段代码替换原 `tmp` 的分配和 kernel 参数；不是在原 kernel 之外再启动一次计算。

参考：NVIDIA Unified Addressing 的主机内存自动映射。复习讲义对应第 2.11、3.5 节。

Q.8 把字节数用于 int 指针偏移

结论：原片段不安全。核心问题是偏移单位。

`size=N*sizeof(int)` 是字节数，但 `dtab + size` 表示前进 `size` 个 int，而不是前进 `size` 个字节。以 CUDA 常见的 `sizeof(int)=4` 为例：

操作	预期元素区间	原代码实际元素区间
stream 0，N 个元素	`[0,N)`	`[0,N)`
stream 1，2N 个元素	`[N,3N)`	`[4N,6N)`
stream 2，3N 个元素	`[3N,6N)`	`[12N,15N)`

`dtab` 只有 `6N` 个 int，`htab` 只有 `7N` 个 int，第三段明显越界。第二段在该例中虽仍位于分配内，却不是题意中的连续第二段。仅更改复制长度不够，**复制和 kernel 参数中的指针偏移都要改**。

```
size_t bytes = (size_t)N * sizeof(int);
cudaMemcpyAsync(dtab, htab, bytes,
                cudaMemcpyHostToDevice, stream[0]);
cudaMemcpyAsync(dtab + N, htab + N, 2 * bytes,
                cudaMemcpyHostToDevice, stream[1]);
cudaMemcpyAsync(dtab + 3 * N, htab + 3 * N, 3 * bytes,
                cudaMemcpyHostToDevice, stream[2]);
kernel<<<grid0, block0, 0, stream[0]>>>(dtab, N);
kernel<<<grid1, block1, 0, stream[1]>>>(dtab + N, 2 * N);
kernel<<<grid2, block2, 0, stream[2]>>>(dtab + 3 * N, 3 * N);
cudaMemcpyAsync(htab, dtab, bytes,
                cudaMemcpyDeviceToHost, stream[0]);
cudaMemcpyAsync(htab + N, dtab + N, 2 * bytes,
                cudaMemcpyDeviceToHost, stream[1]);
cudaMemcpyAsync(htab + 3 * N, dtab + 3 * N, 3 * bytes,
                cudaMemcpyDeviceToHost, stream[2]);
for (int s = 0; s < 3; ++s) cudaStreamSynchronize(stream[s]);
```

保留原题中分配与清零即可；本段替换复制、launch 和等待。三个分段互不相交，不需要跨 stream 事件。原题自身已经在每个对应 D2H 之后同步；不能误判为“只等了第三个 stream”。把等待集中到最后，主要是让结构更清楚。

最后打印 `7N` 个元素不导致 host 越界，因为 host 已分配 `7N`；但只有前 `6N` 个是 GPU 结果，最后 `N` 个仍为初始零值。如果只想输出结果，应打印 `6N` 个。

复习讲义对应第 2.4、4.13 节。

Q.9 默认 stream 保证 GPU 顺序，不保证 CPU 已可用

结论：不安全，有缺少主机等待和未初始化输入两个问题。

两个 `cudaMemcpyAsync` 省略 stream 参数，在 CUDA C++ 接口中默认使用 stream 0；kernel 也在该默认 stream，所以 H2D → kernel → D2H 的 GPU 顺序成立。

但 pinned host buffer 的 D2H 异步复制不保证调用返回时数据已可由 CPU 使用。CPU 随后立刻读取并写入 `htab`，可能与仍在进行的传输冲突。`htab` 还未初始化就用作 H2D 输入，若 kernel 读取输入，则结果不可依赖。

修正顺序：先初始化 host 数据；提交 H2D、kernel、D2H；在 CPU 左移数组之前等待完成。可以等待相应默认 stream，或者更清楚地全部使用一个显式 stream。

```
// Initialize htab before the H2D copy.
memset(htab, 0, size);
cudaMemcpyAsync(dtab, htab, size, cudaMemcpyHostToDevice);
kernel<<<grid, block>>>(dtab, N);
cudaMemcpyAsync(htab, dtab, size, cudaMemcpyDeviceToHost);
cudaStreamSynchronize(0);
for (int i = 0; i < N - 1; ++i) htab[i] = htab[i + 1];
htab[N - 1] = 0;
```

本段替换原题复制之后的相关操作，并把初始化移到 H2D 之前。若已确定其他地方写入了正确输入，可保留该输入而不清零。默认 stream 的特殊 GPU 排序规则不能当作 CPU 等待 D2H 的替代。

参考：NVIDIA API 同步行为。复习讲义对应第 2.11、4.13 节。

Q.10 两类独立错误：越界与跨流数据竞争

结论：不安全。即使修正偏移，仍需修复流间依赖。

第一类：指针单位。`sz_in_bytes` 是字节数，`dtab + sz_in_bytes` 在 `int*` 上前进了过多元素。若 int 为 4 字节，第二段从元素 `4N` 开始，已经超过仅 `2N` 元素的分配。改为 `dtab+N` 和 `htab+N`。

第二类：kernel 与数据段所属 stream 交叉。修正指针后，原结构仍是：

- stream 0：复制第一段 → 计算第二段 → 回传第一段。
- stream 1：复制第二段 → 计算第一段 → 回传第二段。

第一段 kernel 在 stream 1，却依赖 stream 0 的 H2D；第一段 D2H 在 stream 0，却需要等 stream 1 的 kernel。第二段同样存在交叉。这些边没有显式建立。

原题中的第一次 `cudaStreamSynchronize(stream[0])` 位于第一段 D2H **之后**，只能等待已经提交的操作结束，不能倒过来让那次 D2H 正确等待 stream 1 的生产者。最后两次等待也无法撤销已经发生的竞争。一次实验碰巧输出正确，不构成安全证明。

更直接的修正：让每个 stream 负责同一段的完整流水线。

```
size_t bytes = (size_t)N * sizeof(int);
for (int s = 0; s < 2; ++s) {
    size_t offset = (size_t)s * N;
    cudaMemcpyAsync(dtab + offset, htab + offset, bytes,
                    cudaMemcpyHostToDevice, stream[s]);
    kernel<<<grid, block, 0, stream[s]>>>(dtab + offset, N);
    cudaMemcpyAsync(htab + offset, dtab + offset, bytes,
                    cudaMemcpyDeviceToHost, stream[s]);
}
for (int s = 0; s < 2; ++s) cudaStreamSynchronize(stream[s]);
for (int i = 0; i < 2 * N; ++i) printf("%d", htab[i]);
```

保留原题的 `2N` 分配和 host 清零，本段替换复制、launch 和等待。若必须保留交叉分工，需在各生产者后记录 event，并在对应消费者前等待；统一每段的负责 stream 更容易证明正确。

复习讲义对应第 2.11、4.13 节。

IV CUDA 求和归约

Q.11 共享内存归约和两次调用

题意：原数据已在 GPU，每个线程读取一个元素，先求每块部分和，再归约部分和。

设第一阶段 `G` 个块、每块 `B` 个线程，`N=G*B`；`G`、`B` 均为 2 的幂。每个线程把自己的值写入 shared memory，随后按 `B/2, B/4, ..., 1` 的步长两两相加。

```
__global__ void reduce_kernel(float *in, float *out)
{
    extern __shared__ float partial[];
    unsigned int t = threadIdx.x;
    size_t i = (size_t)blockIdx.x * blockDim.x + t;
    partial[t] = in[i];
    __syncthreads();

    for (unsigned int stride = blockDim.x / 2;
         stride > 0; stride /= 2) {
        if (t < stride)
            partial[t] += partial[t + stride];
        __syncthreads();
    }
    if (t == 0) out[blockIdx.x] = partial[0];
}
```

屏障放在 `if (t < stride)` 外面，保证整个 block 的线程都能到达。每个 stride 的读取依赖上一轮结果，因此不能随意删掉屏障。每个 block 只写自己的输出位置，不需要跨 block 原子加。

```
// tab is already allocated and initialized on the GPU.
// G and B are positive powers of two and N == G * B.
// Both B and G must fit the device's per-block resource limits.
float *block_sums = nullptr;
float *total = nullptr;
cudaMalloc((void **)&block_sums, (size_t)G * sizeof(float));
cudaMalloc((void **)&total, sizeof(float));

reduce_kernel<<<G, B, (size_t)B * sizeof(float)>>>(tab, block_sums);
cudaGetLastError();
reduce_kernel<<<1, G, (size_t)G * sizeof(float)>>>(block_sums, total);
cudaGetLastError();

float host_sum = 0.0f;
cudaMemcpy(&host_sum, total, sizeof(float), cudaMemcpyDeviceToHost);
cudaFree(total);
cudaFree(block_sums);
```

第三个 launch 参数是每块动态 shared memory 的字节数。第二次 launch 使用 1 个块、`G` 个线程，读取 `G` 个部分和并输出一个总和。两次 launch 在同一 stream 中，第二个 kernel 自然等第一个完成；CPU

中间不必加 `cudaDeviceSynchronize()`。最终同步 D2H 在返回后提供总和。真实程序须检查每次 API 与 launch 的返回/错误状态；以上片段为书面答案。

题目隐含的资源条件：“块数是 2 的幂”不能保证 `G` 个线程能放进一个合法 block。以上两次调用还要求 `G`、`B` 均满足设备线程数和 shared memory 限制。如果第一阶段块数过大，应多级归约；或改变第二阶段设计，让单块线程各自先累加多个部分和。不能无条件用 `<<<1,G>>>` 覆盖任意数据规模。

小例子：`N=16`、`G=4`、`B=4`，输入为 `1...16`。第一阶段得到 `[10,26,42,58]`，第二阶段得到 `136`。浮点加法顺序改变后，通常不保证与串行累加逐位相同；验证应使用有限值检查与合适的绝对/相对误差容限。

V 用这张卷调整复习顺序

以下是这份试卷考过的题型。3 小时是 2025-2026 原卷的时长。

先练什么	对应原题	已有复习入口
数据位置、pinned 与 mapped 的差别	Q1、Q7	第 2.4、2.11、3.5 节
合并访存与线程展平	Q2、Q3	第 1.10、1.11、2.7、4.2 节
CUDA 属性与指令对应	Q4、Q5	第 2.1、2.3、3.13 节及本文 Q5
画依赖、用 events 排序	Q6	第 2.11、4.13 节及本文 Q6
逐行审查字节、元素、区间、完成时机	Q7-Q10	本文 III；不要只背判断结论
shared memory 树形归约	Q11	本文 IV

建议先不看解析做一遍法语题目，再按四个问题审查每段代码：指针属于哪种内存？长度和偏移分别是什么单位？生产者完成后消费者才会执行吗？CPU 什么时候可以读取结果？最后手写一次 Q6 和 Q11，把每条依赖和每次屏障的原因说清楚。